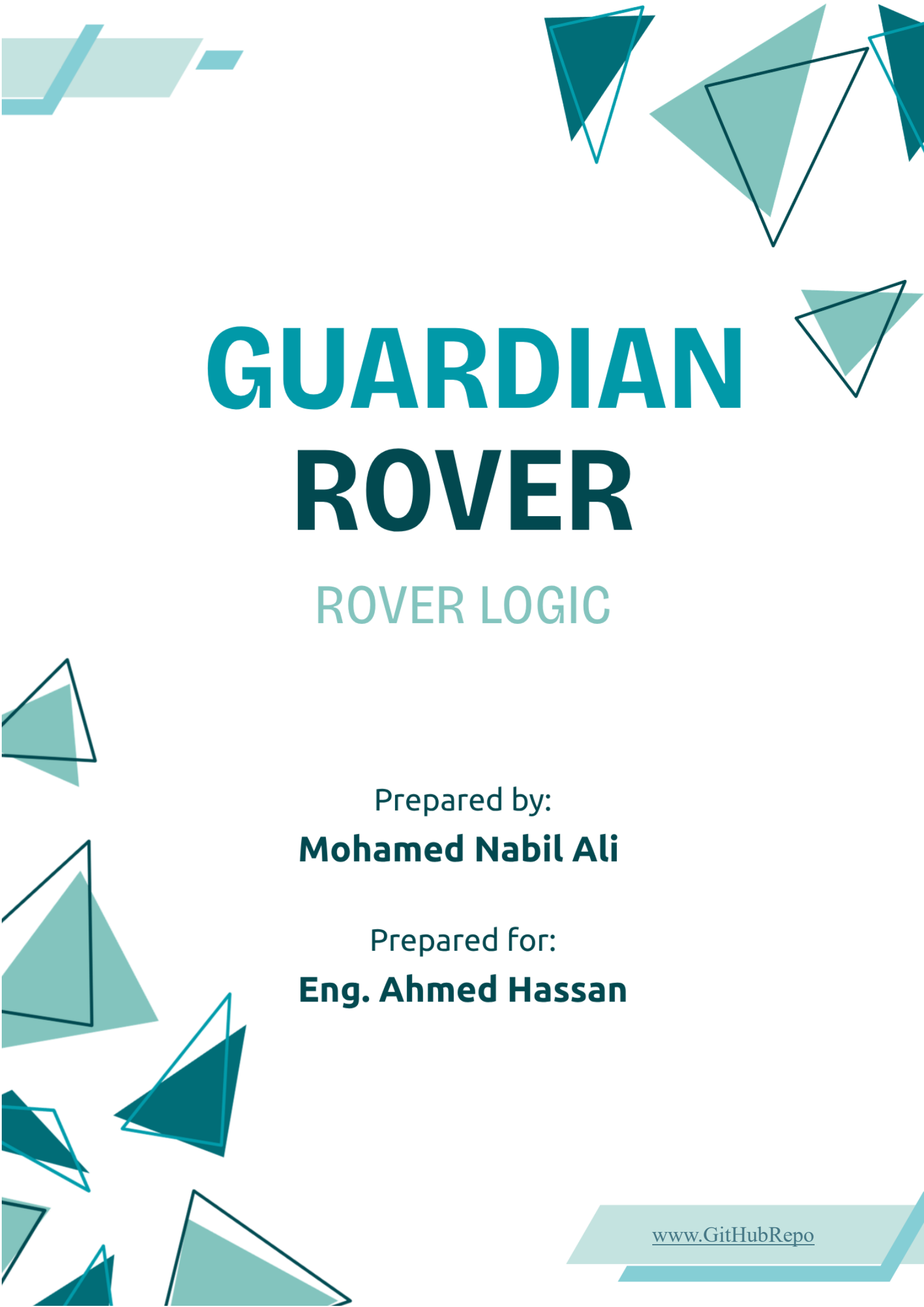


The background of the slide is decorated with various teal-colored geometric shapes, including triangles and polygons, some of which are outlined in a darker shade of teal. These shapes are scattered across the top, bottom, and sides of the slide.

GUARDIAN ROVER

ROVER LOGIC

Prepared by:
Mohamed Nabil Ali

Prepared for:
Eng. Ahmed Hassan

Contents

Introduction	3
Objective.....	4
Sensor Processing Architecture.....	5
Maneuver and Decision Logic	6
Emergency Override Protocol	7
Command Routing and Execution	8
Distance Extraction and Diagnostic Logging	9
Human-Readable State Translation.....	10
Top-Level System Integration and Signal Flow	11
Data Logging Formats: To Workspace Options	12
Control Script and Output	13

Introduction

The Guardian Rover project focuses on the development of a reactive, autonomous navigation system capable of interpreting environmental data and executing safe maneuvers in real-time. Reliable obstacle avoidance, signal filtering, and deterministic state management are critical for unmanned ground vehicles operating in dynamic environments.

This report details the Simulink-based control architecture engineered to manage the rover's behavior. The system architecture begins by processing raw voltage inputs from three ultrasonic sensors, translating them into physical distance metrics and preliminary obstacle flags. To prevent erratic maneuvering caused by sensor noise or transient spikes, these raw flags are processed through an S-Function debounce filter, ensuring an obstacle is stable across multiple execution cycles before the system reacts.

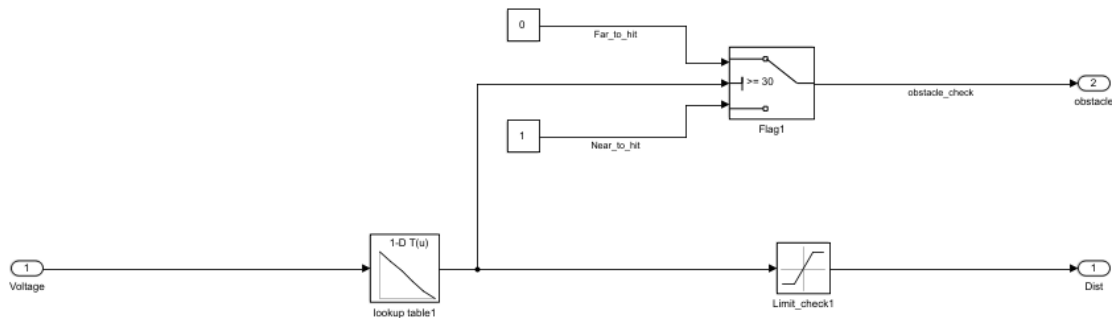
The filtered data is then fed into the autonomous decision node, which calculates the optimal path based on available clearance and outputs a specific state code. This state code is routed through a switch-case command structure to execute the final maneuver. The architecture also features a triggered emergency latch to force an immediate stop under critical conditions. To ensure modularity and clean signal routing across these various subsystems, the model utilizes structured Bus objects for sensor arrays and emergency states. Finally, the system's performance is monitored by a discrete diagnostic subsystem, which captures time-stamped telemetry data for post-simulation analysis and verification.

Objective

The primary objective of this project is to design, simulate, and analyze a modular autonomous control system for the Guardian Rover using MATLAB and Simulink. Specifically, the project aims to:

- Implement robust signal processing for three directional ultrasonic sensors (Front, Left, and Right) using 1-D Lookup Tables and saturation limits.
- Develop a noise-resilient obstacle detection mechanism by routing raw sensor flags through a continuous-execution debounce filter.
- Construct an autonomous decision-making matrix (using an Enabled Subsystem and MATLAB Function) that evaluates spatial clearance to dictate discrete rover maneuvers (Forward, Turn Left, Turn Right, Stop, and Idle).
- Design a dedicated 100ms Function-Call Diagnostics subsystem to log critical system data efficiently without impeding the main control loop.
- Establish a master MATLAB initialization script to load workspace parameters, programmatically construct Bus objects, and automatically generate diagnostic plots post-simulation.
- Evaluate and document the architectural differences and optimal use cases for data routing components (MUX vs. BUS) and data logging formats (Timeseries vs. Array/Structure).

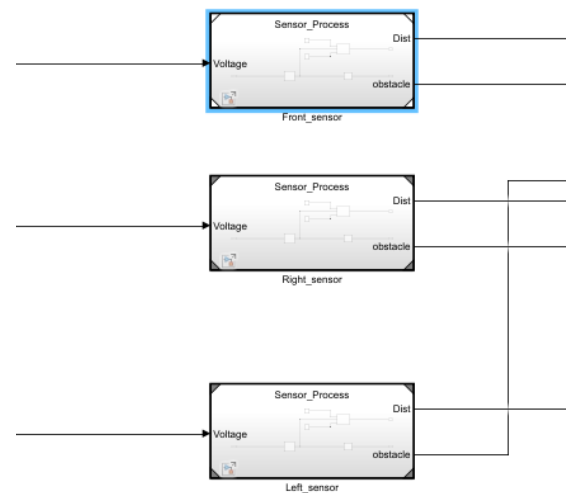
Sensor Processing Architecture



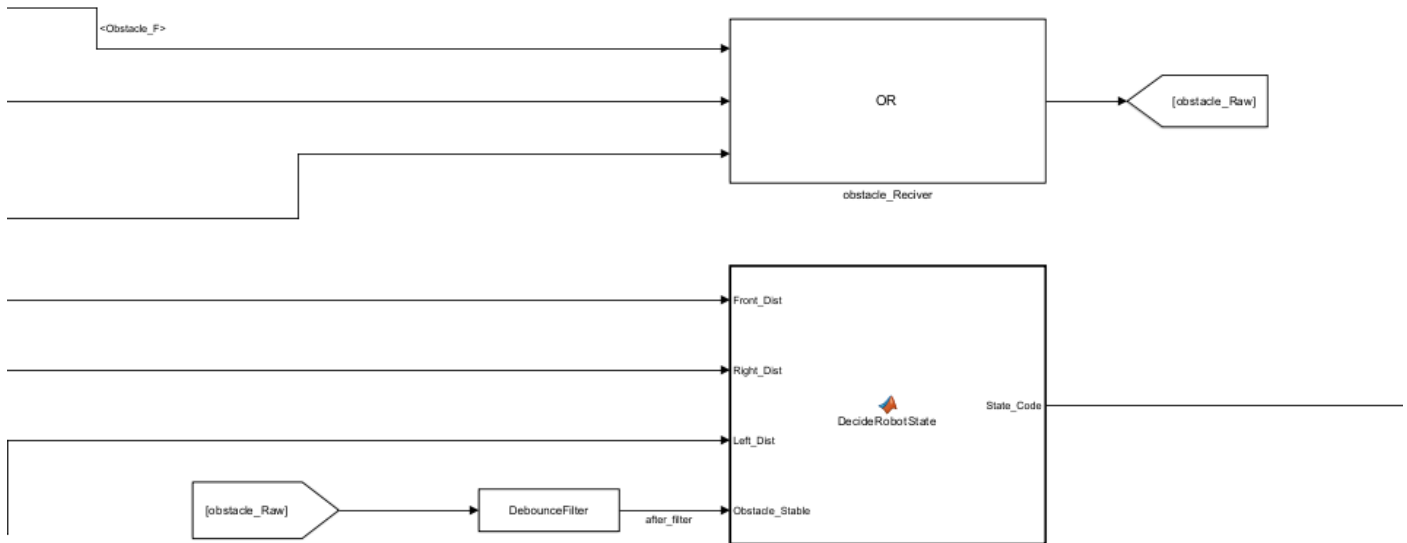
The control system utilizes three identical sensor processing modules dedicated to the Front, Right, and Left ultrasonic sensors. Each of these modules ingests a raw analog Voltage reading and translates it into two distinct system outputs: a continuous distance measurement (Dist) and a binary collision warning flag (obstacle).

Internally, the Sensor_Process block relies on a dual-path logic structure to generate these outputs:

- **Voltage-to-Distance Conversion:** The raw input voltage is first fed into a 1-D Lookup Table (lookup table1), which mathematically interpolates the electrical signal into a physical spatial distance.
- **Conditioned Distance Output (Dist):** The converted distance value is routed through a saturation limit block (Limit_check1). This ensures the final output remains clamped within a physically valid and safe operational range before entering the main data bus.
- **Proximity Threshold Flag (obstacle):** The interpolated distance is simultaneously fed into the control port of a logical Switch block (Flag1). The switch evaluates this distance against a hardcoded threshold of 30. If the distance is greater than or equal to 30, the switch outputs a 0 (Far_to_hit), indicating the path is clear. If the distance falls below 30, it outputs a 1 (Near_to_hit), actively flagging the presence of an obstacle.



Maneuver and Decision Logic

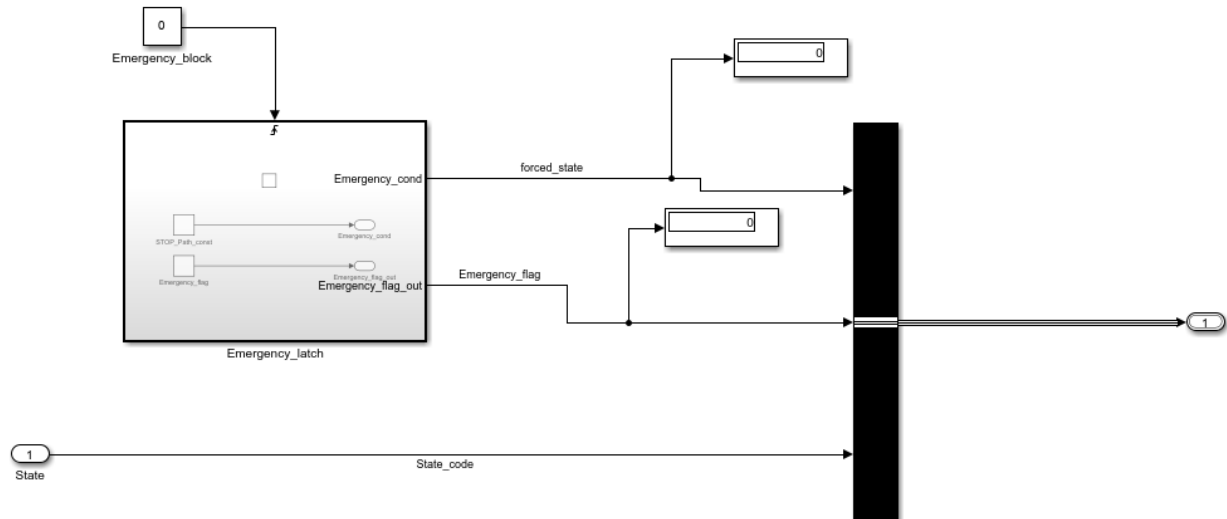


The core decision-making routing is housed within the Maneuver subsystem. This section of architecture is responsible for aggregating sensor data, filtering out transient noise, and computing the final autonomous state command.

The logic flows through three primary stages:

- **Obstacle Flag Aggregation:** The individual obstacle detection flags from the sensors (such as <Obstacle_F>) are routed into a logical OR gate labeled obstacle_Reciver. If any of the connected sensors detect an object, this gate triggers a combined [obstacle_Raw] signal.
- **Signal Stabilization:** To prevent erratic robot behavior caused by temporary sensor glitches, the [obstacle_Raw] signal is passed through a DebounceFilter. This block processes the raw input and outputs an after_filter signal, ensuring that the obstacle detection is stable before the robot reacts.
- **State Calculation:** The final autonomous determination is executed by a MATLAB Function block named DecideRobotState. This block evaluates the stabilized obstacle flag (ingested as Obstacle_Stable) alongside the continuous spatial data from all three sensors (Front_Dist, Right_Dist, and Left_Dist). Based on this combined telemetry, the function outputs a definitive State_Code to dictate the rover's next movement.

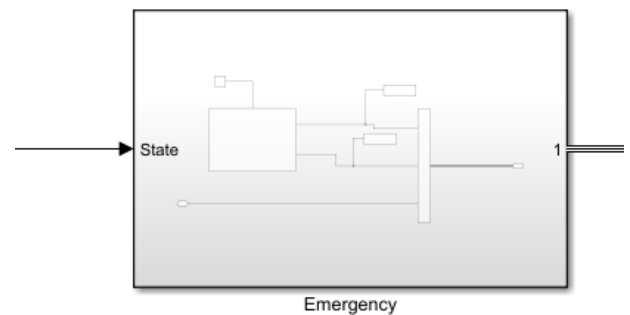
Emergency Override Protocol



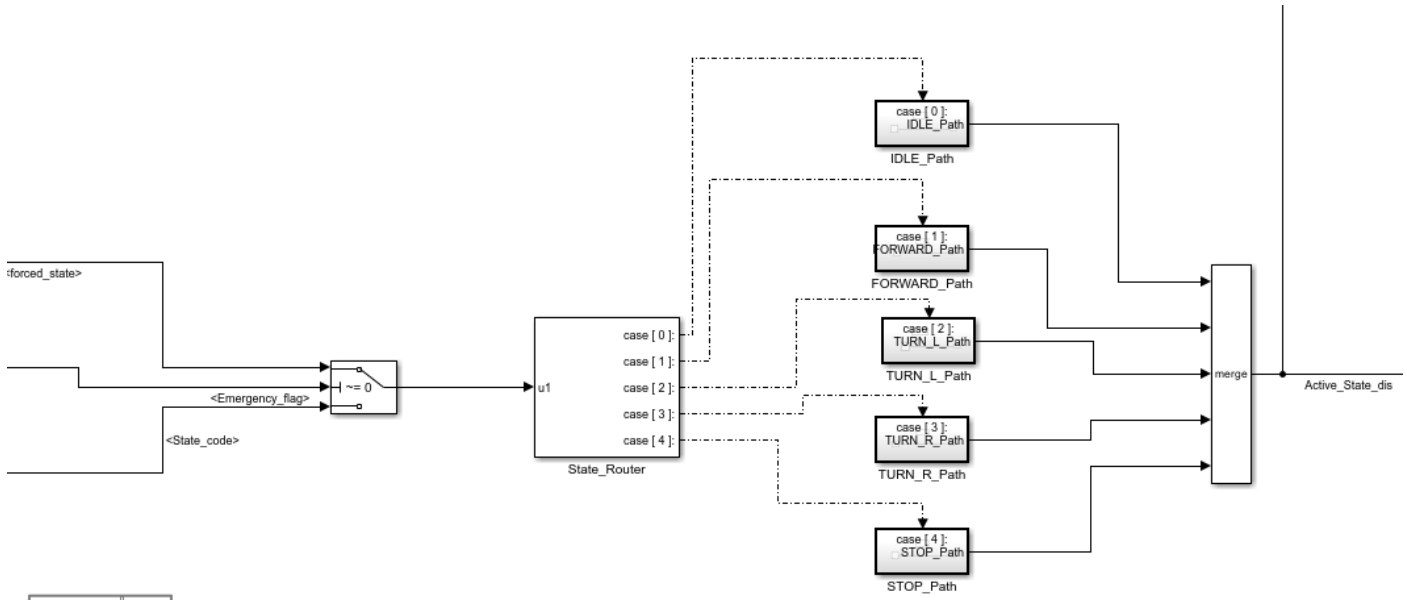
To ensure operational safety, the control architecture features an overriding Emergency subsystem positioned directly between the autonomous decision node and the final state router. This block acts as a fail-safe mechanism capable of preempting the standard maneuver logic.

The core of this subsystem is the Emergency_latch, configured as a Triggered Subsystem. Its functionality is defined as follows:

- **Trigger Input:** The latch is activated by an external boolean control signal, currently simulated using a constant block labeled Emergency_block.
- **Forced State Generation:** When triggered, the internal logic forces the system into a predefined safe state. It outputs two specific signals: a forced_state code (set to STOP) and an Emergency_flag boolean to indicate that an override is actively occurring.
- **Bus Integration:** These two emergency signals are combined with the standard autonomous State_code (which flows untouched through the bottom of the subsystem) into a unified EmergencyStateBus via a Bus Creator block. This bundled signal is then routed out to the system's command output stage, where a Switch block determines whether to execute the standard state or enforce the emergency stop based on the status of the Emergency_flag.



Command Routing and Execution



The final stage of the control architecture is the State_code subsystem, which handles the resolution of emergency overrides and the physical routing of maneuver commands.

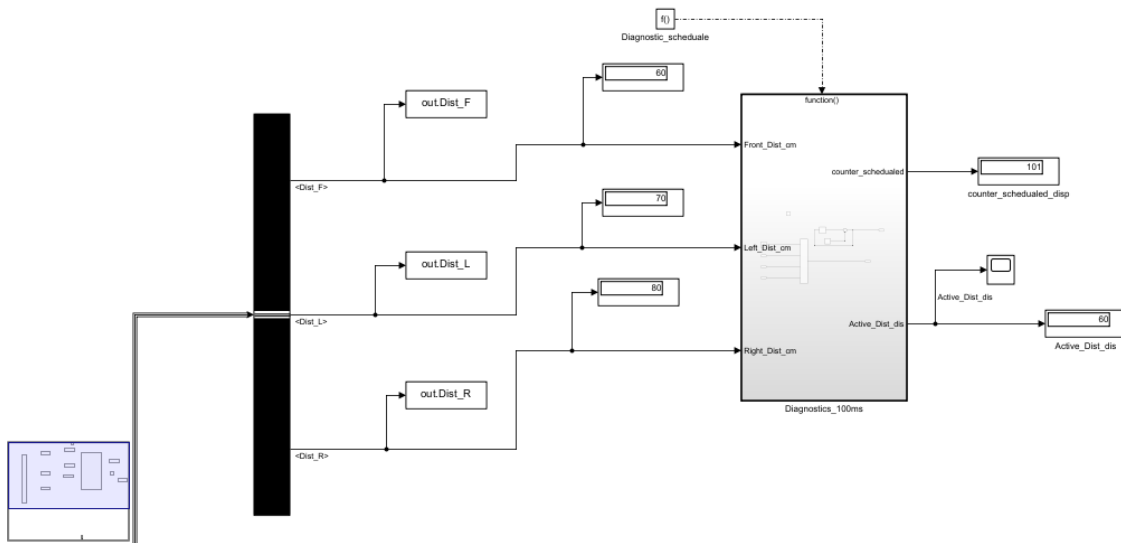
The incoming EmergencyStateBus is first unpacked using a Bus Selector, extracting the <forced_state>, <Emergency_flag>, and standard <State_code> signals. These signals are fed into a Switch block that acts as the final safety gatekeeper. The Switch evaluates the <Emergency_flag>; if the flag is active (non-zero), it overrides the system and passes the <forced_state>. If the flag is inactive, it permits the standard autonomous <State_code> to pass through.

The selected state integer is then transmitted to the State_Router, a Switch Case block. This router acts as an execution dispatcher, triggering one of five mutually exclusive action subsystems based on the evaluated integer value:

- case [0] activates IDLE_Path
- case [1] activates FORWARD_Path
- case [2] activates TURN_L_Path
- case [3] activates TURN_R_Path
- case [4] activates STOP_Path

Finally, the outputs from these individual path subsystems are routed into a central Merge block. Because only one switch case subsystem executes at any given time, the Merge block safely consolidates the active signal into a single Active_State_dis output, which is then broadcast to the system diagnostics.

Distance Extraction and Diagnostic Logging

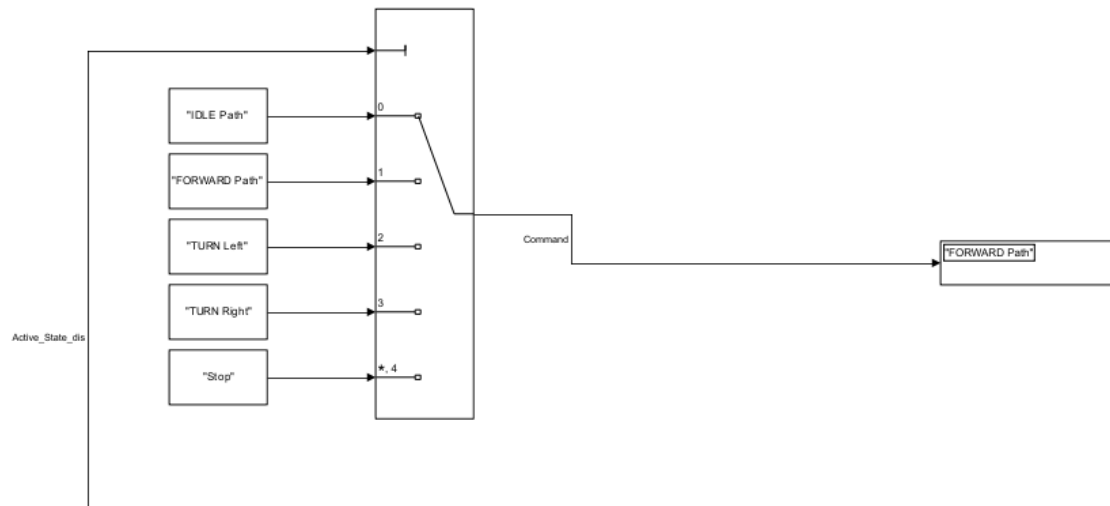


The diagnostic data logging module extracts specific sensor metrics from the bundled system data. A Bus Selector unpacks the individual distance measurements for the front (<Dist_F>), left (<Dist_L>), and right (<Dist_R>) sensors. These unpacked signals are split into three simultaneous paths:

- They are captured by "To Workspace" blocks (out.Dist_F, out.Dist_L, out.Dist_R) for post-simulation MATLAB analysis.
- They are visualized directly on numerical display blocks, which currently show static test values of 60, 70, and 80.
- They are routed into the Diagnostics_100ms subsystem.

This diagnostic subsystem is triggered by a Diagnostic_schedule function-call, meaning it executes at a dedicated time interval rather than continuously. It evaluates the incoming distances to output an active distance metric (Active_Dist_dis), currently displaying 60, and tracks the number of execution cycles using a counter_scheduled display.

Human-Readable State Translation



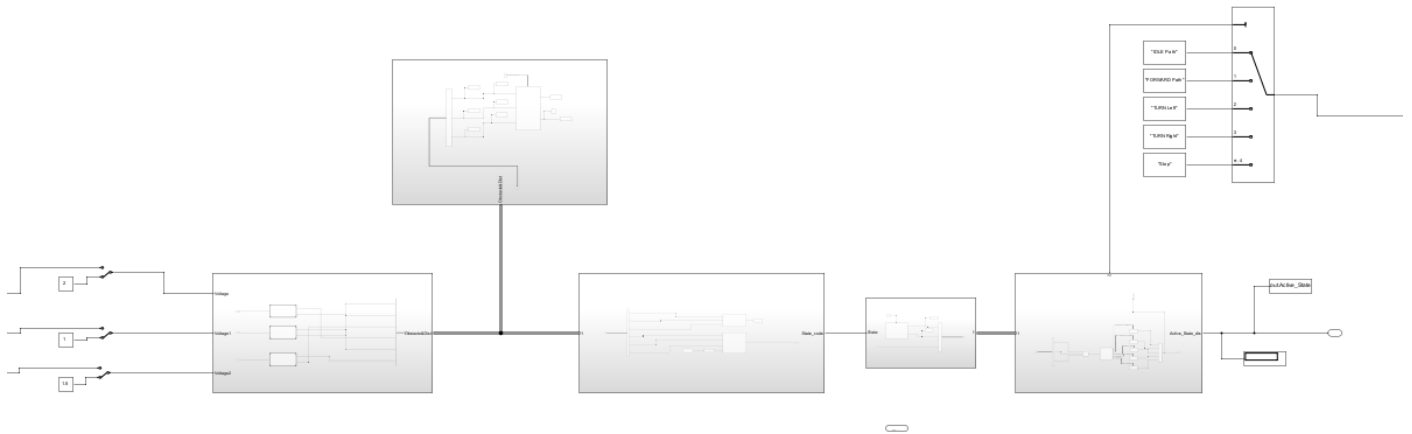
To translate the raw numerical state commands into readable operational feedback during simulation, the system utilizes a dedicated status display stage. The integer control signal `Active_State_dis` drives a Multiport Switch that maps the numeric outputs (0 through 4) to distinct string constants.

The block maps the values as follows:

- 0 connects to "IDLE Path"
- 1 connects to "FORWARD Path"
- 2 connects to "TURN Left"
- 3 connects to "TURN Right"
- 4 connects to "Stop"

The selected text is output via the `Command` signal to a visual display block. This setup allows operators to immediately verify the rover's active maneuver—currently showing "FORWARD Path"—without needing to manually decode the integer values.

Top-Level System Integration and Signal Flow



The top-level Simulink architecture integrates the individual subsystems into a cohesive, unidirectional signal flow from raw input to final executed command.

The system begins at the input stage, utilizing Manual Switches to allow operators to toggle between dynamic external sensor feeds and static constant test values (currently set to 2, 1, and 1.8). These selected signals feed into the Voltage, Voltage1, and Voltage2 ports of the primary sensor processing block.

Once processed, the individual distance metrics and obstacle flags are consolidated into a unified, thick-line bus named Obstacle&Dist. To maintain a clean top-level canvas, this bus branches into two distinct paths:

- **Diagnostic Path:** The bus routes upward into the function-call triggered diagnostics subsystem, ensuring all raw telemetry is available for the 100ms logging cycle without cluttering the main control loop.
- **Control Path:** The main branch continues rightward into the autonomous decision block (port 1), which evaluates the telemetry and outputs a discrete numerical State_code.

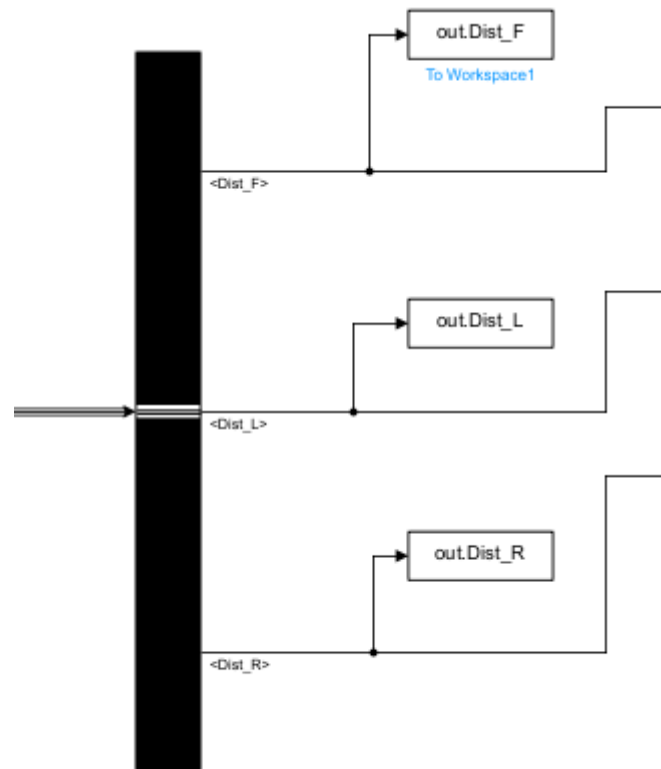
The State_code signal passes sequentially through the Emergency block, which can inject an override flag, before entering the final command routing subsystem. This router outputs the final resolved integer command as Active_State_dis.

In the final stage, the Active_State_dis signal splits to drive three distinct visualization and logging endpoints: it is captured by a "To Workspace" block (out.Active_State), displayed numerically on the canvas, and routed upward into the multiport switch array to trigger the human-readable string translation (e.g., "FORWARD Path").

Data Logging Formats: To Workspace Options

The To Workspace block is utilized in the diagnostics subsystem to export simulation data to the MATLAB base workspace for post-processing and visualization. Simulink offers several save formats, each suited for different use cases:

- **Array:** Saves the signal data as a standard N-dimensional MATLAB matrix. This format is lightweight and useful for simple mathematical operations, but it strips away time data. If an Array is used, the time steps must be logged separately (e.g., via a clock block) to plot the data over time accurately.
- **Structure:** Saves the data as a MATLAB struct containing two main fields: time and signals (which includes values and dimensions). This keeps the timestamps and data bundled together and is useful when logging multidimensional signals.
- **Timeseries (Selected Format):** This format wraps the data values, timestamps, and signal metadata (like units or names) into a single MATLAB timeseries object. This was the format chosen for the Guardian Rover project because it allows for clean, object-oriented data manipulation in the master control script. By using Timeseries, the script can call `simOut.Active_State.Time` and `simOut.Active_State.Data` directly, ensuring the diagnostic plots are perfectly synchronized with the simulation clock.



Control Script and Output

To ensure a streamlined testing and validation process, the entire Simulink architecture is governed by a master MATLAB script (run_MAIN.m). This script serves as the centralized control hub, establishing a programmatic link between the MATLAB base workspace and the Simulink environment.

The script executes the system sequentially through four primary phases:

```
%% Guardian Rover - Master Initialization and Control Script
clear; clc; close all;

%% 1. Create Bus Objects
% Run your existing scripts to load the buses into the workspace
run('EmergencyState.m');
run('SensorDataBus.m');

% Define Lookup Table data
sensor_voltage_breakpoints = [0 1 2 3 4 5];
sensor_distance_table = [100 80 60 40 20 5];

disp('Parameters and Bus Objects loaded successfully.');
```

```
%% 3. Run Simulation
model_name = 'MAIN';
load_system(model_name);

disp('Running simulation...');
simOut = sim(model_name);
disp('Simulation complete.');
```

- **Bus Object Initialization:** The script first calls two external definition files, EmergencyState.m and SensorDataBus.m. This programmatically builds the required Bus objects and loads them into the workspace, ensuring the Simulink model has the correct structural definitions for signal routing before execution.
- **Parameter and Gain Loading:** To avoid hardcoding values deep inside the Simulink block parameters, the script defines critical system variables such as dist_threshold_stop, dist_threshold_turn, and debounce_limit. It also loads the calibration arrays for the sensor 1-D Lookup Tables. By defining these in the master script, operators can rapidly tune the rover's behavior without manually navigating through subsystem block dialogs.

- **Automated Simulation Execution:** The script utilizes the `sim()` command to automatically load and execute the `MAIN.slx` model, capturing the entire simulation output into a single `simOut` data object.
- **Diagnostic Visualization:** Following the simulation, the script extracts the structured Timeseries data generated by the model's "To Workspace" blocks. It automatically plots this data into a two-panel figure. The top subplot displays the discrete step-changes of the `Active_State` over time, while the bottom subplot provides a continuous overlay of the Front, Left, and Right sensor distances. This automated visualization provides immediate validation of the autonomous decision logic against the raw sensor telemetry.

